OnlineCourseCompare: Database Setup Guide

The votes database (“Worth it or money grab?”) • one-time setup • November 4, 2026

What this is

The website’s courses are built into the website itself. This database only stores the anonymous votes. If it is ever paused or deleted, **the site keeps working**; only voting stops until you set it up again.

The setup script is saved in the website code at `online-course-next/supabase/setup.sql` (and on GitHub), and printed in full at the end of this guide. Running it on any new, empty Supabase project rebuilds the same database in about a minute.

1. Create the project

1. Go to `supabase.com/dashboard`, open your **free** organization, and click **New project**.
2. Name: `onlinecoursecompare`. Database password: click **Generate** and save it in your password manager.
3. Region: **East US (North Virginia)**. The website runs in Vercel’s Washington, D.C. region, so this keeps votes fast.
4. Click **Create new project** and wait about 2 minutes for it to start.

2. Run the setup script

1. In the left menu, open **SQL Editor** and click **New query**.
2. Paste the whole script: copy it from `supabase/setup.sql` or from the last pages of this guide.
3. Click **Run**. You should see **Success. No rows returned**.
4. Check: open **Table Editor**. You should see an empty table called `course_votes`.

Running the script twice is safe: it only creates what is missing. Supabase’s Security Advisor may say **RLS Enabled No Policy** for `course_votes`. That is on purpose: only the website’s server is allowed to read or write votes.

3. Connect the website

1. In Supabase, click **Connect** at the top of the project page (or go to **Project Settings > API Keys**). Copy the **Project URL** and a **Secret key** (it starts with `sb_secret_`; click **Create new API keys** if there isn’t one).
2. In Vercel, open the `online-course-next` project, go to **Settings > Environment Variables**, and add:

```
SUPABASE_URL         = the Project URL
SUPABASE_SECRET_KEY  = the secret key
```

3. Redeploy: **Deployments** > the latest one > **...** > **Redeploy**.

Keep the secret key secret. Never paste it into code, GitHub, a chat, or a variable that starts with `NEXT_PUBLIC_`. If it leaks, delete it under **API Keys** and make a new one. The website code that uses these two variables is added in Phase 2, so this step only matters once that is live.

4. Keeping it free and safe

- **Pausing:** Supabase pauses free projects after about a week with almost no database activity, and emails you a warning first. Phase 2 adds a daily check from the website that keeps it active.
- **If it gets paused:** open the dashboard, click the project, then **Resume project**. This works for 90 days after the pause, with all votes kept.
- **After 90 days:** you can still download a backup from the project page, or start fresh: make a new project and run the setup script again.
- **Back up the votes** (once a month is plenty): in **Table Editor**, open `course_votes` and export it as CSV. Keep the file somewhere safe.

To restore votes into a new project:

1. Create the project and run the setup script (steps 1 and 2).
2. In **Table Editor**, open `course_votes`, choose **Insert > Import data from CSV**, and pick your backup file.
3. In **SQL Editor**, run this line once so new votes don't clash with the restored ones:

```
select setval(pg_get_serial_sequence('public.course_votes', 'id'),
              (select coalesce(max(id), 0) + 1 from public.course_votes), false);
```

5. Handing it to Claude (or anyone) later

Give them this guide and access to the GitHub repo, and say something like:

"Set up the OnlineCourseCompare votes database: run online-course-next/supabase/setup.sql in my new Supabase project, then add SUPABASE_URL and SUPABASE_SECRET_KEY to the Vercel project and redeploy."

- If Claude has a Supabase connection to that organization, it can run the script for you. If not, you paste it into the SQL Editor yourself (step 2); it is the same result either way.
- **Moving it to another account:** Supabase can move a project between organizations (**Project Settings > General > Transfer project**). You need to be the owner of the organization it is in now, and a member of the one it is going to.

The setup script

Same as `online-course-next/supabase/setup.sql`. Copy everything in the grey box (it may continue on the next page).

```
-- ======================================================================
-- OnlineCourseCompare: database setup for "Worth it or money grab?" votes
--
-- Run this once in a new Supabase project:
--   Supabase dashboard -> your project -> SQL Editor -> New query
--   -> paste this whole file -> Run
--
-- Safe to run again: it only creates what is missing and changes nothing
-- else. It creates the structure only. Votes people have already cast are
-- data; back those up separately (see the setup guide PDF).
--
-- The course list itself is NOT in the database. It lives in the website
-- (src/data/courses.json), so the site keeps working even if this
-- database is paused or gone. Only voting depends on it.
-- ======================================================================
-- One row per visitor per course. Nobody signs in. voter_hash is a one-way
-- hash the website's server makes from the visitor's connection, so a
-- visitor can change their vote but can't vote twice, and no IP address
-- is ever stored.
create table if not exists public.course_votes (
  id          bigint generated by default as identity primary key,
  course_id   bigint      not null,   -- "id" from src/data/courses.json
  platform    text        not null,   -- 'Udemy', 'Coursera', ...
  verdict     text        not null check (verdict in ('worth_it', 'money_grab')),
  voter_hash  text        not null,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now(),
  constraint course_votes_one_per_voter unique (course_id, voter_hash)
);

create index if not exists course_votes_platform_idx
  on public.course_votes (platform);
-- Totals the website shows: per course, and per platform.
create or replace view public.course_vote_totals
with (security_invoker = true) as
select
  course_id,
  count(*) filter (where verdict = 'worth_it')   as worth_it,
  count(*) filter (where verdict = 'money_grab') as money_grab,
  count(*)                                       as total
from public.course_votes
group by course_id;

create or replace view public.platform_vote_totals
with (security_invoker = true) as
select
  platform,
  count(*) filter (where verdict = 'worth_it')   as worth_it,
  count(*) filter (where verdict = 'money_grab') as money_grab,
  count(*)                                       as total
from public.course_votes
group by platform;
```

```
-- Lock it down. The public key can't read or write votes; only the
-- website's server, using the project's secret key, can.
alter table public.course_votes enable row level security;

revoke all on public.course_votes         from anon, authenticated;
revoke all on public.course_vote_totals   from anon, authenticated;
revoke all on public.platform_vote_totals from anon, authenticated;

-- Newer Supabase projects no longer give new tables API access by
-- default, so give the secret key (service_role) exactly what it needs.
grant select, insert, update on public.course_votes      to service_role;
grant select on public.course_vote_totals, public.platform_vote_totals
  to service_role;

-- Tell the API to pick up the changes right away.
notify pgrst, 'reload schema';
```